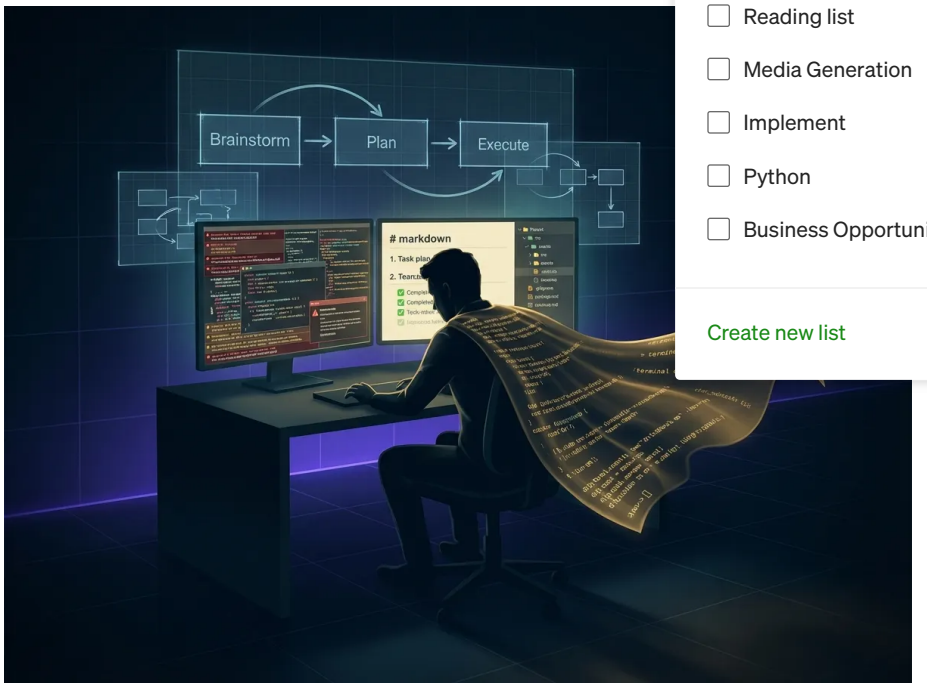
I Gave Claude Code a Brain. It's Called Superpowers — and It Has 150,000 GitHub Stars for a Reason

Anil Mathew Follow 9 min read · Apr 14, 2026



I asked Claude Code to build a feature last month.

It jumped straight into writing code. No questions, no spec, no plan. Within 3 minutes it had changed 11 files — and broken 6 tests that had nothing to do with my request.

That's when I found Superpowers. And nothing has been the same since.

The Dirty Secret About AI Coding Agents

Most developers using Claude Code, Cursor, or Copilot have experienced this: you describe a feature, the agent generates confident-looking code, and then you spend the next hour untangling decisions you never agreed to.

The AI isn't dumb. It's actually *too fast*. It skips the part that matters most — the thinking.

A senior engineer, before writing a single line, would ask: *What are we actually building? What are the edge cases? What could go wrong? Is there a simpler way?*

AI coding agents, by default, don't do this. They optimise for tokens written, not for outcomes delivered.

As one widely-shared LinkedIn comment put it: *"82,000 developers just agreed on something: the biggest problem with AI coding is not intelligence. It is discipline."*

Superpowers fixes this. And it has 150,000 GitHub stars to show that a lot of developers felt the same pain.

What Is Superpowers, Exactly?

Superpowers is an open-source agentic skills framework and software development methodology built by Jesse Vincent (of Prime Radiant). It's a plugin for Claude Code — and also works with Cursor, Codex, OpenCode, and Gemini CLI.

It's a structured set of instructions, skills, and workflow triggers that make your coding agent behave like a disciplined software engineer, not an impatient intern.

The plugin launched on October 9, 2025 — the same day Anthropic released the Claude Code plugin system. It was accepted into the official Anthropic Claude Code marketplace on January 15, 2026. As of April 2026: 150,000 stars, 13,000 forks, 28 contributors — one of the fastest-growing open-source repos of the year.

Simon Willison, creator of Datasette, called Jesse Vincent *"one of the most creative users of coding agents"* he knows, and described the ideas in Superpowers as genuinely fascinating.

Part 1: How to Install Superpowers in Claude Code

Prerequisites

Make sure you have:

- Claude Code 2.0.13 or later (`claude --version` to check)
- An active Anthropic account (Pro or Max plan)
- Claude Code authenticated via `claude login`

To update Claude Code if needed:

bash

```
npm install -g @anthropic-ai/claude-code
```

Method 1: Official Anthropic Marketplace (Recommended)

Superpowers is in the official Anthropic plugin marketplace. One command inside Claude Code:

```
/plugin install superpowers@claude-plugins-official
```

Quit and restart Claude Code. At the start of your next session, you'll see an injected prompt that begins:

```
<session-start-hook><EXTREMELY_IMPORTANT>
You have Superpowers.
```

This bootstrap is what activates the entire skills system automatically.

Method 2: Community Marketplace

If Method 1 doesn't work on your version:

```
/plugin marketplace add obra/superpowers-marketplace
/plugin install superpowers@superpowers-marketplace
```

Method 3: Other Platforms

Cursor:

```
/add-plugin superpowers
```

Or search "superpowers" in the Cursor plugin marketplace.

Gemini CLI:

bash

```
gemini extensions install https://github.com/obra/superpowers
```

Codex / OpenCode: Tell the agent directly —

```
Fetch and follow instructions from https://raw.githubusercontent.com/obra/superp
```

Verify It's Working

Start a new session and type `/help` — you should see Superpowers commands listed. Or just say *"Help me plan this feature"* and watch what happens. If Claude asks you questions instead of immediately writing code, the plugin is active.

Part 2: How to Use It — The Slash Commands

Superpowers mostly works automatically — skills trigger based on what you're doing. But knowing the explicit commands gives you precise control over the workflow.

The 3 Commands You'll Use Every Day

`/superpowers:brainstorm`

Start here for any non-trivial feature. Claude enters a Socratic design dialogue — targeted questions, exploration of alternatives, hidden requirements surfaced. The session ends with a design document you sign off on before a single line of code is written.

Usage:

```
/superpowers:brainstorm I want to build a real-time notification system for our
```

`/superpowers:write-plan`

After brainstorming (or with an existing design doc), this generates a complete implementation plan — not a vague outline. Exact file paths, specific code, verification commands, success criteria, and a rollback plan.

One developer used this before a Next.js caching migration and received a 500-line plan covering all 23 API route files that needed changes, 2 components using `new Date()` that would break prerendering, specific context providers needing Suspense boundaries, and a full 4-day timeline with testing checkpoints.

`/superpowers:execute-plan`

Runs the approved plan using subagent-driven development. Fresh subagent per task. Two-stage review after each task (spec compliance first, then code quality). Claude can work autonomously for 1–2 hours on complex features without context drift.

Full Command Reference

CommandWhen to Use `/superpowers:brainstorm` Before any complex feature `/superpowers:write-plan` After design sign-off, or for migrations/refactors `/superpowers:execute-plan` Run the plan in batches with checkpoints `/using-superpowers` If Claude drifts and "forgets" its skills mid-session `/superpowers:debug` Systematic 4-phase root cause debugging `/superpowers:code-review` Pre-merge review against spec and code quality

Note: Skills can also be triggered conversationally — *"Use superpowers to help me brainstorm this task"* works just as well as a slash command, which is useful mid-flow when you can't remember the exact syntax.

Part 3: The 7-Stage Workflow That Changes Everything

When Superpowers is active, every feature request goes through a 7-stage pipeline. Each stage gates the next — you cannot skip ahead.

Stage 1 — Brainstorming: Socratic dialogue. Design document. Your sign-off required before anything else.

Stage 2 — Git Worktrees: Isolated branch created automatically. Clean test baseline verified. Your main branch is untouched.

Stage 3 — Writing Plans: 2–5 minute tasks with exact file paths, complete code, and verification steps for each.

Stage 4 — Subagent-Driven Development: Fresh subagents per task. No context drift. Two-stage review per task. 1–2 hours of autonomous work without going off the rails.

Stage 5 — Test-Driven Development: Strict RED-GREEN-REFACTOR. Write failing test → watch it fail → write minimal code → watch it pass → commit → refactor. Code written before a failing test exists gets *deleted*. Test coverage typically hits 85–95%.

Stage 6 — Code Review: Issues classified by severity between tasks. Critical issues block progress.

Stage 7 — Branch Finishing: Options presented on completion: merge, PR, keep, or discard. Worktree cleaned up.

Part 4: 12 Community-Tested Tips and Tricks

These come from real developer experiences shared across Hacker News, GitHub discussions, and developer blogs — patterns that emerged from thousands of developers running Superpowers in production.

Tip 1: Give brainstorming a specific context line

Don't just type `/superpowers:brainstorm`. Give it an anchor prompt:

```
/superpowers:brainstorm I want to build a prototype for [specific issue]. Help m
```

The more specific your opening line, the sharper the questions Claude asks back. Vague input → vague questions → weak design doc.

Tip 2: Manually edit the plan before executing it

After `/superpowers:write-plan`, don't immediately run execute. Open the generated plan doc, review every task, and make manual corrections where Claude made wrong assumptions. The cycle `write-plan → edit → execute` produces dramatically better results than trusting the plan blindly.

Tip 3: Clear context between plan writing and execution

One of the most-cited community tips: after your plan is written and reviewed, start a *fresh Claude Code session* before running execute-plan. This prevents accumulated context from the brainstorming phase polluting the clean subagents during execution.

Tip 4: Use `/using-superpowers` when Claude drifts

In long sessions, Claude can gradually "forget" that it has Superpowers skills and start skipping steps. Typing `/using-superpowers` re-activates the skills dispatcher and resets it back to the structured workflow. Think of it as a reset button for the methodology.

Tip 5: Encode your codebase conventions as custom skills

The `writing-skills` skill lets you create your own SKILL.md files for your specific project. If your codebase has naming conventions, architectural rules, or domain-specific patterns — encode them. Claude will follow them automatically in every session, just like the built-in skills.

Jesse Vincent on this: *"It was a matter of describing how I wanted the workflows to go… and then Claude put the pieces together."*

Tip 6: Give it a programming book

One of the more creative community uses: give Claude a technical book (DDD, clean architecture, system design, etc.) with the prompt: *"Read this book and pull out reusable skills that weren't obvious to you before."* The resulting custom skills encode domain expertise directly into your agent's behaviour — permanently.

Tip 7: Run a second Claude as a staff engineer reviewer

Before running execute-plan, open a second Claude Code session and ask it to review the plan *as a staff engineer*. Cross-agent review catches assumptions your first session normalised. Some developers go further and use cross-model review — having Gemini or GPT-4o review a Claude-generated plan before execution begins.

Tip 8: Use brainstorming for debugging, not just new features

The community has found the brainstorming skill highly effective for diagnosing complex bugs. Start a brainstorm session describing the bug and what you've already tried. The Socratic dialogue often surfaces the root cause *before* any code changes are made. Cheaper and faster than trial-and-error debugging.

Tip 9: Use write-plan as a spec document, not just an execution guide

For migrations, large refactors, or multi-file changes — run `/superpowers:write-plan` even if you plan to execute manually or hand off to a junior developer. The plan becomes a spec, a checklist, and a PR description simultaneously. It also makes code review dramatically easier.

Tip 10: Token efficiency — trust the 5-minute chunk model

The subagent model is significantly more token-efficient than one giant unstructured session. Instead of Claude burning context holding everything in memory, it splits work into 5-minute chunks and writes progress to markdown files. Developers consistently report fewer retries, better first-attempt quality, and lower token burn.

Tip 11: Know its limits — step outside it for environment firefighting

The community is clear on this: Superpowers is built for feature development. Platform-specific debugging, Docker network issues, macOS vs Linux toolchain differences — these fall outside the plan-first workflow. Trying to force structured methodology onto non-plannable problems just adds friction. Recognise when to step outside the workflow and debug directly.

Tip 12: Challenge Claude before it writes the PR

A community favourite: after execution completes, before merging, say: *"Grill me on these changes and don't make a PR until I pass your test."* Or: *"Prove to me this works — diff main against the branch and explain every change."* This catches edge cases that automated review misses and builds your own understanding of what actually shipped.

Why This Matters for Claude Code Users Specifically

Three things Superpowers gets right that most developers overlook:

Context decay is real. In long coding sessions, Claude's context fills up and older decisions get dropped. Subagent-driven development solves this structurally — each subagent starts fresh with exactly the context it needs.

YAGNI is actively enforced. In a world where AI agents love adding abstractions, Superpowers is a constant pull toward simplicity.

Verification before completion. The `verification-before-completion` skill makes Claude actually confirm something works before declaring it done. Sounds obvious. It isn't the default.

Developer Richard Porter described the effect clearly: *"When Superpowers is active, Claude literally cannot skip the planning phase or write code before tests. It behaves less like a code generator and more like a disciplined senior developer who happens to type very fast."*

Getting Started: Your First 10 Minutes

1. Install: `/plugin install superpowers@claude-plugins-official`
2. Restart Claude Code
3. Navigate to your project folder in the terminal
4. Type: `/superpowers:brainstorm [your next feature idea]`
5. Answer Claude's questions honestly — this is where the value is created
6. Review and sign off on the design doc
7. Run: `/superpowers:write-plan`
8. Manually edit the plan where needed
9. Start a fresh session, run: `/superpowers:execute-plan`
10. Sit back while Claude works systematically through every task

The initial brainstorm + plan overhead is 10–20 minutes. The time you save in retries, rework, and cleanup is measured in hours.

The Bottom Line

150,000 GitHub stars in 6 months. One of the fastest-growing open-source repos of 2026. Not because it's clever. Because it solves a real problem that every developer using AI tools has felt: *the model is fast, but undisciplined speed is just expensive waste.*

Superpowers doesn't make Claude smarter. It makes Claude *consistent*. And in production software, consistency beats raw intelligence every single time.

If this was useful, follow me — I write about building real AI-powered products from Kochi, India, including VoIP systems, SaaS platforms, and open-source voice AI infrastructure. One piece a week.

What's the first feature you're going to run through the Superpowers workflow? Drop it in the comments below.

Claude Code | Ai Coding | Claude

Written by Anil Mathew

96 followers · 13 following

Follow

Tech entrepreneur | Co-Founder BitVoice Solutions | Building vexyl.ai - Voice AI platform | VoIP & Unified Communications